# AI Usage Note

Console Agent in C# — ToolForge Agent

Team: ToolForge | Submission: June 2026 | Challenge: AI Prototype (1–2 Days)

AI NOTES

## Overview

This document records every instance of AI-assisted development during the ToolForge Agent project — what was asked, what was generated, where AI fell short, and how the team corrected those failures. It is submitted as a required deliverable per the AI Prototype Challenge rules.

| Metric | Value |
|---|---|
| AI Coding Assistant Used | Claude (claude.ai) + GitHub Copilot |
| Estimated Speed Boost | ~60% faster than writing from scratch |
| AI-generated code kept as-is | ~40% |
| AI-generated code modified | ~45% |
| AI-generated code discarded | ~15% |
| Most valuable AI contribution | ReAct loop scaffold + JSON function schemas |
| Least valuable AI contribution | Business logic & security checks |

## What AI Helped With

| Area | How AI Was Used | File |
|---|---|---|
| Agent Loop Design | Scaffolded the ReAct (Reason + Act) loop pattern in C# — message history management, tool_calls parsing, finish_reason loop exit | AgentLoop.cs |
| Function Calling Schema | Generated OpenAI-compatible JSON tool definitions for all 3 tools (get_weather, get_time, query_sqlite) with correct parameter types | ToolRegistry.cs |
| SQLite Integration | Wrote parameterized async query executor and schema init code with proper using/await patterns | SqliteTool.cs SchemaSetup.cs |
| Prompt Engineering | Drafted initial system prompt with tool docs, behaviour rules, and UTC timestamp injection. Iterated via chat. | PromptBuilder.cs |
| Error Handling | Suggested try/catch patterns for HTTP timeouts, 401/404 API errors, and graceful string-return error contracts | WeatherTool.cs TimeTool.cs |

| Area | How AI Was Used | File |
|---|---|---|
| Unit Tests | Generated xUnit test scaffolding covering happy path, edge cases, and the FakeHttpHandler test double pattern | AgentLoopTests.cs SqliteToolTests.cs |
| README | Drafted architecture overview, setup instructions, sample I/O blocks, and folder structure documentation | README.md |
| Web UI Integration | Generated agent.js fetch wrapper, tokenStats.js pricing tables, and ChatController.cs HTTP bridge | agent.js ChatController.cs |

## ■ What AI Got Wrong — Corrections Applied

| Issue | What AI Generated | How We Fixed It |
|---|---|---|
| Async deadlocks | Used .Result on async Task calls — a classic C# deadlock pattern in ASP.NET contexts | Replaced all .Result/.Wait() with proper await throughout the call chain |
| Tool loop termination | Agent loop didn't check finish_reason == "stop" — ran indefinitely on simple queries | Added explicit break condition; also added a max-iteration guard (5 loops) |
| SQL injection guard | First version only checked if query started with SELECT — still allowed SELECT 1; DROP TABLE | Added full blocklist regex scan for DROP, DELETE, INSERT, UPDATE, ALTER, TRUNCATE, ATTACH |
| Timezone cross-platform | Used Windows-only TimeZoneInfo IDs (e.g. "India Standard Time") — crashes on Linux | Switched to TimeZoneConverter NuGet package which maps IANA ↔ Windows IDs |
| Namespace conflicts | Generated duplicate class names across files causing CS0101 build errors | Reorganised into ToolForge.Agent, ToolForge.Tools, ToolForge.Database namespaces |
| Test isolation | First test version used a shared singleton DB — tests interfered with each other | Each test class creates a fresh temp-file DB via IAsyncLifetime + cleanup on dispose |

## ■ Best Prompts Used During Development

### 1. Agent Loop Bootstrap

***Prompt sent to AI:***

```
You are a C# expert. Build a console AI agent loop that: 1. Sends user input + tool definitions to OpenAI Chat Completions API 2. If the response contains tool_calls, execute the matching C# method 3. Append the tool result as a "tool" role message 4. Loop until finish_reason is "stop" Use async/await throughout. Model: gpt-4o-mini. No external agent frameworks.
```

**Outcome:** This gave us the entire skeleton of AgentLoop.cs. We then corrected the .Result deadlock and added the iteration cap.

### 2. SQL Safety Validation

*Prompt sent to AI:*

```
Write a C# method that validates a raw SQL string before execution on SQLite. Allow only
SELECT statements. Reject any string containing: DROP, DELETE, INSERT, UPDATE, ALTER, CREATE,
TRUNCATE (case-insensitive). Throw InvalidOperationException with a clear message if blocked.
```

**Outcome:** Good first draft. We added ATTACH/DETACH to the blocklist and changed the throw to a return string so the agent loop never crashes.

### 3. OpenAI Function Schema Generator

*Prompt sent to AI:*

```
Generate the OpenAI function-calling JSON tool definition for a C# method: string
GetWeather(string city) The tool should clearly describe what it does, its parameter name,
type, and that it is required. Output raw JSON only.
```

**Outcome:** Saved significant time — generated all three tool definitions in one pass. Required no corrections.

### 4. xUnit Test Scaffolding

*Prompt sent to AI:*

```
Write xUnit tests for a C# class WeatherTool with method GetWeather(string city). Cover: valid
city returns non-null string, empty city throws ArgumentException, invalid city returns a
user-friendly error string (not an exception). Mock HttpClient using a custom FakeHttpHandler.
```

**Outcome:** Very useful for test structure. We extended coverage with Theory/InlineData for all SQL blocklist cases.

### 5. Cross-Platform Timezone Fix

*Prompt sent to AI:*

```
My C# TimeTool.cs crashes on Linux with TimeZoneNotFoundException. I am using
TimeZoneInfo.FindSystemTimeZoneById("India Standard Time"). Fix this to use IANA timezone IDs
like "Asia/Kolkata" on both Windows and Linux. Use the TimeZoneConverter NuGet package.
```

**Outcome:** Diagnosed and fixed within one turn. Exact fix applied as-is to TimeTool.cs.

## ■ Overall AI Assistance Assessment

| Dimension | Rating | Notes |
|---|---|---|
| Speed boost | ★★★★★ | ~60% faster initial scaffolding |
| Boilerplate quality | ★★★★■ | Good; requires namespace and async review |
| Security awareness | ★★■■■ | Missed SQL injection and deadlock patterns — always verify |
| Test generation | ★★★★■ | Strong structure; edge cases needed manual addition |
| Documentation | ★★★★★ | README and comments were production-ready with minor edits |
| Architecture design | ★★★■■ | Suggested patterns; team made all structural decisions |

**Key takeaway:** AI is an excellent pair-programmer for scaffolding, schema generation, and documentation. It is a poor security reviewer. Every AI-generated file was reviewed by a team member before being committed to the

repository.